

[NOME DA INSTITUIÇÃO DE ENSINO]
[CURSO]

Lucas Senderski
RU: 4758862

ATIVIDADE EXTENSIONISTA III
Tecnologia Aplicada à Inclusão Digital — Análise
Evolução e Gerenciamento do Projeto AgroHero

[CIDADE]
2026

ATIVIDADE EXTENSIONISTA III

Tecnologia Aplicada à Inclusão Digital — Análise

Evolução e Gerenciamento do Projeto AgroHero

Acadêmico	Lucas Senderski
RU	4758862
Curso	[preencher: Bacharelado em Engenharia de Software / Ciência da Computação / Sistemas de Informação]
Disciplina	Atividade Extensionista III
Projeto	AgroHero — Marketplace de produtos orgânicos
Repositório	https://github.com/lucassenderski/agrohero
Aplicação publicada	https://agrohero-six.vercel.app
Período de desenvolvimento	15/09/2026 a 29/09/2026
Local	Toledo — Paraná

[preencher: polo / cidade / mês / ano]

Sumário

1. Introdução
 2. O projeto AgroHero
 3. Gerenciamento do projeto
 4. Qualidade do projeto e do software
 5. Métricas do projeto
 6. Evolução do projeto
 7. Implantação
 8. Respostas ao quadro Quem / O quê / Quando / Como / Por quê
 9. Conclusão
 10. Referências
 11. Apêndices
-

1. Introdução

Esta atividade dá continuidade ao projeto **AgroHero**, estruturado na Atividade Extensionista II. O AgroHero é um marketplace de produtos orgânicos que conecta agricultores familiares da região de Toledo, no Oeste do Paraná, a consumidores locais.

A proposta é reduzir a distância entre a produção e o consumo: o produtor cadastra o que colhe, o consumidor compra e retira no local combinado, e o valor é pago presencialmente na retirada, sem intermediário financeiro.

A escolha do tema parte de uma realidade concreta da região. Toledo é um dos maiores polos agroindustriais do Paraná, mas o produtor familiar — aquele que planta em pequena escala, vende na feira e depende de poucos compradores — tem pouca presença digital. A tecnologia, nesse cenário, não é um fim em si mesma: é o meio de dar visibilidade a quem já produz e ampliar o acesso do consumidor a alimento fresco e de origem conhecida.

A Atividade Extensionista III tem como foco **a evolução e o gerenciamento** do projeto. Conforme o enunciado, ela consiste na escolha de, no mínimo, um dos itens propostos para evoluir e gerenciar o software desenvolvido na etapa anterior. Este trabalho adota **cinco** dos itens, descritos na tabela abaixo, por considerá-los os que mais se articulam entre si e os que representam, de fato, o que aconteceu durante o período:

#	Item escolhido	Onde é tratado
1	Definição do processo de gerenciamento	Seção 3.1
2	Controle de configuração, através do uso do GitHub	Seção 3.2
3	Plano de gerenciamento de configuração	Seção 3.3
4	Utilização de métricas para o projeto (SLOC e KSLOC)	Seções 3.4 e 5
5	Qualidade do projeto e/ou do software	Seção 4
6	Evolução do projeto	Seção 6

Além desses, a **implantação do projeto** é tratada na Seção 7. O enunciado a torna obrigatória para os cursos de Bacharelado em Ciência da Computação, Engenharia de Software e Sistemas de Informação, que possuem três Atividades Extensionistas — e é também o desfecho natural de um projeto que já se encontra em produção.

1.1 Objetivo geral

Evoluir o AgroHero a partir do estado em que foi entregue na Atividade Extensionista II, implantando-o em ambiente de produção, e conduzir essa evolução com práticas explícitas de gerenciamento, controle de configuração e garantia de qualidade.

1.2 Objetivos específicos

- Definir e documentar o processo de gerenciamento adotado no projeto;
- Registrar o plano de gerenciamento de configuração, identificando quais artefatos são versionados e quais não são;
- Aplicar métricas objetivas (SLOC, KSLOC, cobertura de testes) para acompanhar o crescimento e a qualidade do software;
- Evoluir o sistema com, no mínimo, uma funcionalidade significativa e corrigir defeitos reais identificados;
- Elevar a qualidade por meio de testes automatizados, cobrindo caminhos negativos e não apenas o caso feliz;

- Implantar a aplicação em produção, com evidências verificáveis do que foi publicado.

1.3 Justificativa da escolha dos itens

O item **"reuso do projeto"** foi descartado de forma consciente. O AgroHero é um sistema web específico para um domínio (marketplace de produtos orgânicos com pagamento na retirada); seus módulos não foram desenhados como biblioteca reutilizável, e extraí-los agora custaria mais do que traria retorno. O enunciado é explícito: *"não escolha o item 'reuso do projeto' se o projeto que está sendo desenvolvido não poderá ser reusado posteriormente"*. Forçar esse item produziria um capítulo artificial.

Os cinco itens escolhidos, por outro lado, descrevem o que efetivamente ocorreu e deixam rastros verificáveis no repositório: o processo aparece nos pull requests, o controle de configuração no histórico do Git, as métricas nos relatórios de cobertura e na contagem de linhas, a qualidade nas suítes de teste, e a evolução nos commits da funcionalidade de pagamento na retirada.

2. O projeto AgroHero

2.1 O que é

O AgroHero é composto por duas aplicações:

- **Backend** — API REST em Node.js/Express que expõe 48 endpoints e conversa com um banco PostgreSQL de 12 tabelas de negócio;
- **Frontend** — aplicação React de página única (SPA), com 22 rotas, que consome a API.

O sistema atende três perfis de usuário:

Perfil	O que faz no sistema
Consumidor	Navega pelo catálogo, filtra por categoria e preço, monta o carrinho, informa o endereço, fecha o pedido e paga na retirada. Avalia produtos que já recebeu.
Agricultor	Cadastra produtos, controla estoque, acompanha os pedidos que contêm itens seus, atualiza o status de cada item e confirma o recebimento do valor na retirada.
Administrador	Gerencia categorias e produtos, modera o catálogo e acompanha os pedidos.

2.2 Setor da sociedade e realidade local

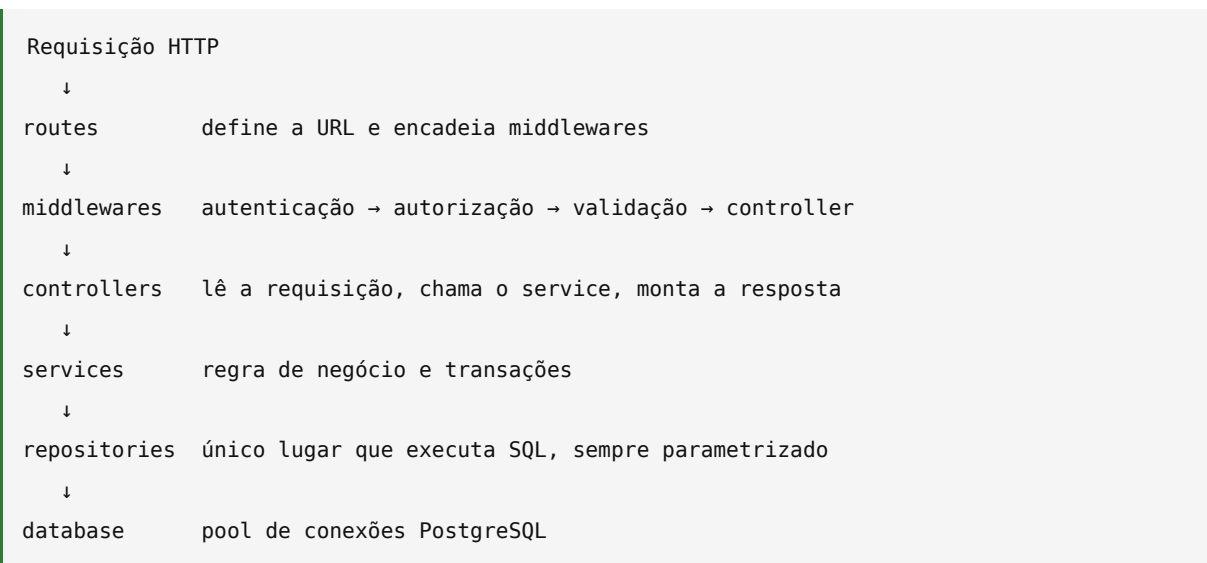
Toledo está entre os maiores municípios produtores do Paraná, com forte presença de agricultura familiar nos distritos do interior. O setor escolhido é o da **agricultura familiar e do consumo local de alimentos orgânicos**. A contribuição social do projeto está em dar ao pequeno produtor uma vitrine digital que ele não teria condições de manter sozinho, e em encurtar a cadeia entre quem planta e quem consome.

Esse recorte orienta decisões concretas de projeto:

- **Não há gateway de pagamento.** A realidade do produtor familiar é a venda na feira, com dinheiro, PIX ou cartão na hora. Um gateway online adicionaria taxas, burocracia (CNPJ, conta de recebimento) e risco de inadimplência a um produtor que vende R\$ 50 por semana. O pagamento é feito na retirada, presencialmente.
- **O frete é calculado no servidor**, com faixa de gratuidade, porque a entrega é feita pelo próprio produtor na cidade.
- **A linguagem e as mensagens de erro são em português**, e os termos usados são os do campo ("safra", "distrito", "propriedade"), não os de um e-commerce genérico.

2.3 Arquitetura

O backend segue uma arquitetura em camadas, com separação estrita de responsabilidades:



A regra de ouro do projeto é: **controller não escreve SQL, repository não decide regra de negócio, service não conhece req/res**. Essa disciplina tem uma consequência prática importante para os testes: o checkout pode ser testado na camada de serviço, sem subir um servidor HTTP, porque o service não depende de nada do Express.

2.4 Tecnologias

Camada	Tecnologias
Backend	Node.js 22, Express 4, PostgreSQL 16, pg, JWT, bcrypt, Zod, Helmet, Pino, Swagger
Frontend	React 18, Vite 5, React Router 6
Banco	PostgreSQL 16 (Docker Compose em desenvolvimento; Neon em produção)
Testes	Jest + Supertest (backend), Vitest + Testing Library (frontend)
Versionamento	Git + GitHub
Implantação	Vercel (frontend), Render (backend), Neon (banco)

2.5 Estado do projeto ao final da Atividade Extensionista II

Ao fim da AE II, o sistema estava funcional e coberto por testes, mas com uma decisão de arquitetura que se mostrou inadequada ao contexto real: o checkout cobrava **online**, por meio de um gateway de pagamento (Mercado Pago, ou um simulador em desenvolvimento). O restante do sistema — catálogo, carrinho, pedidos, avaliações, painéis — já estava implementado e testado.

Esse ponto de partida é o que define a evolução descrita na Seção 6.

3. Gerenciamento do projeto

Esta seção responde aos itens "definição do processo de gerenciamento", "controle de configuração" e "plano de gerenciamento de configuração", além de introduzir as métricas aprofundadas na Seção 5.

O gerenciamento de projetos, na definição usada como referência, é a aplicação de conhecimentos, habilidades, ferramentas e técnicas às atividades do projeto para transformá-lo em realidade com sucesso. No caso de um projeto individual, desenvolvido por uma pessoa com apoio de ferramentas de automação, isso significa estabelecer **um processo explícito** — não confiar na memória nem na improvisação.

3.1 Definição do processo de gerenciamento

O projeto adotou um processo incremental em **fases numeradas (0 a 24)**, cada uma com critério de conclusão verificável. A lista completa está no `README.md` do repositório, o que torna o processo auditável: qualquer pessoa pode conferir em que fase o projeto está e o que falta.

O ciclo de uma unidade de trabalho é:

1. Planejamento → a fase é definida e quebrada em tarefas
2. Implementação → código + testes no mesmo movimento
3. Verificação → suíte completa, lint e build precisam passar
4. Registro → documentação e métricas atualizadas
5. Publicação → commit, pull request, revisão, merge
6. Implantação → deploy e conferência do artefato publicado

Dois elementos merecem destaque porque foram decisões deliberadas de gerenciamento:

(a) A documentação é memória do projeto, não burocracia. O arquivo `AGENTS.md` na raiz do repositório concentra o conhecimento acumulado: convenções que não devem ser quebradas, decisões de banco de dados, segurança, e uma seção chamada **"Armadilhas já encontradas (não repetir)"**, com mais de trinta situações que já custaram tempo e foram diagnosticadas. Isso é gerenciamento de conhecimento: o erro acontece uma vez, é registrado com o sintoma e a causa, e não se repete. Exemplos reais registrados:

- *"O painel do Render diz o deploy disparado, não o que está no ar."* — a conferência confiável é o artefato (hash do bundle), não o painel.
- *"Zod descarta campo não declarado, em silêncio."* — causa de um bug real em que a localização do produtor era gravada vazia.
- *"Rota literal precisa ser declarada antes da rota com parâmetro."* — `GET /produtos/meus` capturado como `:id`.

(b) A definição de pronto inclui verificação, não só implementação. Uma funcionalidade só é considerada pronta quando: os testes passam, o caminho negativo está coberto, o lint está limpo, o build funciona e a documentação (OpenAPI, README) está sincronizada. Há um teste automatizado que **garante** que a especificação OpenAPI documenta exatamente as rotas implementadas — nenhuma a mais, nenhuma a menos. Isso impede que a documentação "apodreça em silêncio".

3.2 Controle de configuração com GitHub

O controle de configuração é exercido por meio do Git e do GitHub. Além do armazenamento do código, o repositório é usado ativamente para **gerenciar mudanças**:

Estratégia de ramificação. O trabalho acontece em branches temáticas (`feat/`, `fix/`, `docs/`), nunca direto na `main`:

```
main                ← sempre estável, é o que está em produção
├─ feat/pagina-sobre → página "Sobre a iniciativa"
├─ fix/spa-fallback   → correção do 404 em rotas profundas
├─ fix/remove-diagnostico-home
├─ docs/registra-sobre-spa
└─ ...
```

Fluxo de integração. Todo trabalho passa por pull request, mesmo sendo um projeto individual. A justificativa é prática: o pull request cria um **ponto de revisão explícito**, com diff isolado, descrição do problema e da solução, e um registro permanente do porquê da mudança. Em um projeto individual, esse é o mecanismo que substitui a revisão por par.

Foram **11 pull requests**, todos revisados, aprovados e integrados por merge:

PR	Título	Arquivos	Linhas
#9	fix: fallback SPA no Vercel para as rotas do React Router	1	+4 / -0
#8	feat: página "Sobre a iniciativa"	10	+338 / -22
#6	fix: tira os dados de infraestrutura da front page	4	+37 / -100
#4	feat: status da API e crédito do autor no rodapé	6	+165 / -4
#2	feat: página de receitas + pagamento na retirada, sem gateway online	97	+8497 / -2453
#1	Agents/frontend style update request	—	—

Marcos de configuração (linha de base). O projeto mantém uma **tag anotada** que marca a versão em produção no fim da AE II:

```
v1.0.0-producao → commit c2a97a3
```

Essa tag é o ponto de rollback documentado: se um deploy futuro der errado, é para ela que se volta. O procedimento está na seção 9 de `docs/DEPLOY.md`.

Versionamento do banco. O schema não é alterado manualmente. Ele é definido por **10 migrations numeradas e idempotentes** (001 a 010), aplicadas em ordem, cada uma registrando sua própria mudança. Isso significa que o estado do banco é reprodutível a partir do código — um ambiente descartável pode ser integralmente recriado com `npm run migrate && npm run seed`. As migrations carregam comentários extensos explicando **por que** a mudança foi feita, e não apenas o que ela faz (a 008, por exemplo, tem 40 linhas de comentário antes da primeira instrução).

Nada de segredo no repositório. O arquivo `.env` está fora do Git. Há uma guarda em `config/env.js` que **recusa subir a aplicação em produção** se o `JWT_SECRET` contiver valores de exemplo (`troque`, `placeholder`, `changeme`, `example`) ou se a origem de CORS usar `*` ou `http://`. Falha fechada: o processo é encerrado na subida, em vez de operar de forma insegura e silenciosa.

3.3 Plano de gerenciamento de configuração

O plano define, para cada categoria de artefato, se ele é versionado, como é identificado e quem o controla.

Artefato	Versionado?	Identificador	Local
Código-fonte (backend e frontend)	Sim	Git	backend/src/, frontend/src/
Testes automatizados	Sim	Git	backend/tests/, frontend/src/testes/
Migrations do banco	Sim	Numeração 001 – 010	backend/src/database/migrations/
Seeds (dados iniciais)	Sim	Git	backend/src/database/seeds/
Especificação da API (OpenAPI)	Sim	Git, com teste de sincronia	backend/src/docs/openapi.js
Documentação do projeto	Sim	Git	README.md, AGENTS.md, docs/
Configuração de infraestrutura	Sim	Git	docker-compose.yml, render.yaml, frontend/vercel.json
Definição de dependências	Sim	package.json + package-lock.json	Raízes de cada app
Variáveis de ambiente e segredos	Não	—	Painel do provedor (Render)
Dependências instaladas	Não	—	node_modules/ (ignorado)
Artefatos de build	Não	—	dist/ (ignorado)

Regras de identificação. Cada artefato versionado é identificado pelo commit SHA. As releases são identificadas por tag anotada. Cada build do frontend é identificado pelo **hash de conteúdo** dos arquivos gerados pelo Vite (index-D6UwP-jn.js), o que permite provar qual commit está publicado — detalhe explorado na Seção 7.

Controle de mudança. Toda mudança segue: branch temática → commit descrito em português no imperativo → pull request → merge. As mensagens de commit seguem o padrão tipo: descrição (feat, fix, docs, test, refactor, chore). A distribuição real, no repositório:

Tipo	Commits
feat (nova funcionalidade)	25
docs (documentação)	16
fix (correção de defeito)	6
test (testes)	4
refactor	1
chore	1

Controle de versão do banco em produção. As migrations são aplicadas automaticamente no `startCommand` do Render (`npm run migrate && npm start`). O encadeamento com `&&` preserva uma propriedade essencial: **se a migration falhar, o servidor não sobe e a versão anterior continua no ar**. A falha é segura por construção.

3.4 Métricas selecionadas

O projeto adota métricas objetivas, detalhadas na Seção 5: **SLOC** (Source Lines of Code), **KSLOC** (milhares de linhas), cobertura de testes por camada, contagem de testes e tamanho das mudanças por pull request. A finalidade não é medir produtividade por volume de linhas — isso seria uma métrica enganosa — mas acompanhar o **crescimento do sistema ao longo das revisões** e verificar se a qualidade acompanha o crescimento.

4. Qualidade do projeto e do software

O item "qualidade" foi tratado em três frentes: **testes automatizados, segurança e gestão de defeitos**.

4.1 Testes automatizados

A decisão central de qualidade do projeto é **não usar mocks**. Os testes de integração rodam contra um PostgreSQL real e uma API real. Isso foi uma escolha com custo (mais lentos, exigem o banco no ar) e com benefício comprovado: **bugs reais foram descobertos justamente porque o teste exercitou o caminho verdadeiro**.

As suítes, no estado atual:

Suíte	Arquivos	Testes	Ferramentas
Backend	25	630	Jest + Supertest
Frontend	11	62	Vitest + Testing Library
Total	36	692	

No backend, os testes se dividem em 19 suítes de integração e 6 de unidade. No frontend, são testes de integração de verdade — eles chamam a API real e verificam o estado no banco depois, sem substituir o `fetch`.

Cobertura de caminhos negativos. A convenção do projeto é cobrir não só o caso feliz, mas o que pode dar errado: acesso indevido a recurso de outro usuário (IDOR), requisição sem permissão, valores inválidos, estoque insuficiente, endereço de terceiro, preço manipulado. Há arquivos dedicados a isso (`autorizacao.test.js`, `seguranca.test.js`, `multiagricultor.test.js`).

4.2 Cobertura de testes

O backend mantém um relatório de cobertura executável (`npm run test:coverage`). O resultado geral:

Métrica	Cobertura
Instruções	83,09 %
Ramificações	74,63 %
Funções	83,94 %
Linhas	83,71 %

Por camada, os números mostram onde a cobertura é forte e onde há espaço para evoluir:

Camada	Instruções	Observação
Controllers	96,73 %	Alta, como esperado: são camadas finas
Services	91,29 %	Onde está a regra de negócio — a cobertura cobre o que importa
Repositories	89,01 %	SQL parametrizado
Utils	90,52 %	Validadores e helpers
Middlewares	78,43 %	<code>requireRole</code> pronto mas não usado por nenhuma rota
Database	5,42 %	Scripts de migration/seed, executados fora dos testes

O número da camada `database` (5,42 %) não é uma falha de qualidade: são os scripts de linha de comando (`run-migrations.js`, `run-seeds.js`), que não fazem sentido sob teste unitário. Já o caso do `requireRole` é um **falso baixo**: é um middleware completo e testado, mas a verificação de propriedade acontece dentro dos services, que já carregaram o recurso. Ele permanece como utilitário, com teste de unidade que garante que não apodreça.

4.3 Segurança

A segurança é tratada como requisito de qualidade, com um checklist explícito por fase:

Item	Como é garantido
IDOR	O proprietário de um recurso é sempre resolvido a partir do token , nunca do corpo ou do parâmetro da requisição
Papéis	<code>requireRole</code> em toda rota sensível
Preço/quantidade/estoque	Recalculados no servidor; o preço não existe no schema de entrada
SQL	Sempre parametrizado
Segredos	Fora do Git; guarda de produção recusa valores de exemplo
Logs	Nunca registram senha, token ou dado de cartão
Dados de cartão	A tabela <code>pagamentos</code> não possui nenhuma coluna de cartão

Duas decisões merecem destaque, porque são de modelagem e não de validação:

(a) A melhor defesa é não ter o campo. No carrinho, o schema **não declara** o preço, e a tabela `carrinho_itens` **não tem** coluna de preço — só `quantidade`. O preço somente

pode vir de `produtos.preco`. A manipulação de preço deixa de ser um caso a tratar e passa a ser uma impossibilidade do modelo. O mesmo vale para o checkout, cujo corpo aceita apenas `endereco_id` e `metodo_pagamento`: não há `valor_total` a ser forjado.

(b) 404, não 403, para recurso de outro usuário. Um 403 confirmaria que o recurso existe, permitindo enumerar identificadores e descobrir quem avaliou o quê. O 404 não distingue "não existe" de "não é seu" — o comportamento correto para recurso privado.

4.4 Gestão de defeitos

Durante o período, defeitos reais foram encontrados, diagnosticados e corrigidos — cada um com registro da causa. Três exemplos:

Defeito 1 — Preço do concorrente vazando entre produtores. O `GET /pedidos/:id` devolvia `valor_total` para o produtor, revelando quanto **outro** produtor vendeu no mesmo pedido. Correção: a visão do agricultor passou a **remover** os campos `valor_produtos`, `valor_frete` e `valor_total`, devolvendo apenas `valor_dos_meus_itens`. Enviar os dois valores seria pior — bastaria uma tela usar o campo errado.

Defeito 2 — Mensagem de erro genérica mascarando a causa. O `ErroApi` do frontend guardava o texto em `.message`, mas toda a interface lia `falha.mensagem`. Resultado: `ULTIMO_ENDERECO`, `ENDERECO_PRINCIPAL`, `ESTOQUE_INSUFICIENTE` caíam todos na mensagem genérica "Ocorreu um erro". O usuário nunca via o motivo real. Correção: alias `this.mensagem = mensagem`.

Defeito 3 — Ingrediente casando dentro de outra palavra. A receita de sopa oferecia geleia no lugar do mel, porque o termo `mel` casava em "Frutas Vermelhas". Como a função devolvia o primeiro produto que casava, o falso positivo ainda escondia o produto certo. Correção: comparação por palavra inteira, com expressão regular ancorada.

Esses três casos ilustram um princípio adotado no projeto: **um teste só tem valor depois de verificado que ele falha sem a correção**. Ao corrigir um defeito, o teste é rodado com a correção revertida para confirmar que ele realmente captura o problema. Há registro explícito de um caso em que isso não foi feito e o teste passava "por motivo errado".

5. Métricas do projeto

Esta seção aprofunda o item de métricas (SLOC e KSLOC).

5.1 O que é medido

Métrica	Definição adotada
Linhas físicas	Todas as linhas do arquivo, incluindo em branco e comentários
SLOC	<i>Source Lines of Code</i> — linhas efetivas de código, excluindo brancas e de comentário
KSLOC	SLOC dividido por mil
Cobertura	Percentual de instruções/ramificações exercitadas pelos testes
Tamanho de PR	Linhas adicionadas/removidas e arquivos tocados

A distinção entre linhas físicas e SLOC importa neste projeto porque a documentação e os comentários são volumosos de propósito (migrações comentadas, decisões registradas). Contar linhas físicas inflaria a métrica.

5.2 SLOC atual, por categoria

Categoria	Arquivos	Linhas físicas	SLOC
Backend — código (<code>src/</code>)	86	15.701	9.743
Backend — testes	27	10.668	7.345
Backend — SQL (migrations/seeds)	11	1.006	437
Frontend — código (<code>.jsx/ .js</code>)	66	8.875	6.865
Frontend — CSS	14	3.247	2.549
Frontend — testes	13	2.381	1.511
TOTAL	217	41.878	28.450
TOTAL sem testes	177	28.829	19.594

Em KSLOC: **28,45 KSLOC** no total, ou **19,59 KSLOC** de código de produção (sem contar os testes).

Três leituras interessantes desses números:

- **A proporção de testes é alta.** Os SLOC de teste representam quase metade do total. Em um projeto com forte ênfase em qualidade, essa proporção é esperada e desejável.
- **O CSS não é desprezível** (2.549 SLOC). Não é um sistema com biblioteca de componentes pronta; grande parte da interface foi escrita à mão.
- **Os testes do backend (7.345 SLOC) superam o código de backend em linhas por arquivo**, porque cada cenário de teste precisa montar fixtures próprias — uma consequência direta da decisão de não usar mocks.

5.3 Crescimento ao longo das revisões

A tabela abaixo mede o SLOC em marcos do histórico do Git, mostrando a evolução concreta do projeto desde o primeiro commit até o estado atual:

Marco	Arquivos	Linhas físicas	SLOC
Commit inicial (estrutura)	30	1.692	1.107
Banco de dados	42	3.292	2.148
Backend base	49	4.262	2.745
v1.0.0-producao (fim da AE II)	190	36.254	24.905
PR #2 (pagamento na retirada)	212	41.559	28.223
PR #4 (status no rodapé)	215	41.716	28.324
PR #8 (página Sobre)	218	41.954	28.489
HEAD (PR #10)	218	41.954	28.489

O recorte da **Atividade Extensionista III** (c2a97a3...HEAD) é:

Métrica	Valor
Commits na AE III	40 (dos 64 totais)
Arquivos alterados	108
Linhas adicionadas	+9.109
Linhas removidas	−2.561
SLOC no início (AE II)	24.905
SLOC no fim (AE III)	28.489
Crescimento	+3.584 SLOC (+14,4 %)

O crescimento de 14,4 % em SLOC veio acompanhado de um salto grande de funcionalidade (pagamento na retirada, avatar, logo, página institucional) e de uma **redução** de complexidade em um ponto importante: o gateway de pagamento foi removido. Os −2.561 de linhas removidas são, em boa parte, código de integração com gateway que deixou de existir.

5.4 Métricas de qualidade associadas

Métrica	Valor
Testes backend	630 (25 suítes)
Testes frontend	62 (11 suítes)
Total de testes	692
Cobertura de instruções (backend)	83,09 %
Cobertura da camada de serviços	91,29 %
Endpoints documentados (OpenAPI)	48
Rotas do frontend	22
Tabelas de negócio	12
Migrations	10
Pull requests integrados	11
Tags de release	1 (v1.0.0-producao)

5.5 Interpretação

As métricas, lidas em conjunto, contam uma história coerente: o projeto **cresceu em funcionalidade** (+14,4 % de SLOC), **manteve a qualidade** (cobertura de 83 % e 692 testes passando) e **reduziu complexidade** onde era possível (remoção do gateway). O número absoluto de SLOC, isolado, não diria nada — o que informa é a sua relação com a cobertura de testes e com o tipo de mudança feita.

Uma leitura honesta dos limites dessas métricas: SLOC mede volume, não valor. A remoção de 2.561 linhas (código de gateway) **melhorou** o projeto, e uma métrica de produtividade que premiasse linhas escritas teria penalizado essa melhoria. Por isso as métricas foram usadas para acompanhar tendências e dimensionar o sistema, não para avaliar desempenho.

6. Evolução do projeto

Esta é a parte central da atividade. A evolução do AgroHero na AE III se organiza em torno de uma mudança estrutural e de um conjunto de evoluções incrementais.

6.1 Mudança principal: pagamento na retirada (PR #2)

O problema

Na versão entregue ao fim da AE II, o checkout **cobrava online**. O fluxo era: o cliente fechava o pedido → o sistema chamava um gateway de pagamento → o gateway devolvia o resultado por webhook → o pagamento tinha status `PENDENTE/APROVADO/RECUSADO` e guardava o identificador da transação externa.

Para o público real do projeto, esse desenho era inadequado:

- O produtor familiar vende na feira, com dinheiro, PIX ou cartão **na hora da entrega**;
- Um gateway online impõe taxas sobre vendas pequenas;
- Exigiria do produtor uma conta de recebimento compatível;
- Introduziria o risco de o cliente pagar e não retirar, ou retirar e o pagamento não compensar;
- O sistema dependia de um serviço externo e de um webhook para funcionar.

A decisão de evolução

O gateway foi **removido**, e o pagamento passou a acontecer **presencialmente na retirada**, confirmado pelo produtor pelo próprio painel. Essa decisão reorganizou o modelo de dados em quatro pontos, implementados na migration `008_pagamento_na_retirada.sql`:

- 1. Não há gateway.** As colunas `identificador_externo` (id da transação no provedor) e `resumo_gateway` (resposta resumida) perderam sentido e foram removidas.
- 2. O pagamento passa a ser por produtor, não por pedido.** Um pedido pode conter itens de vários produtores, e cada um recebe o seu na retirada. Com uma linha por pedido, a pergunta "quem confirma o recebimento?" não teria resposta — e um produtor poderia confirmar o pagamento do produto de outro. A posse é o eixo da segurança: o agricultor é resolvido a partir do **token**, e a consulta é por `(pedidoId, agricultorId)`.
- 3. Os métodos passam a ser os do balcão:** `PIX`, `CARTAO`, `DINHEIRO`. `BOLETO` sai (não se compensa um boleto na retirada) e `SIMULADO` sai junto com o gateway de teste.
- 4. O status perde `RECUSADO` e `REEMBOLSADO`.** Não há recusa possível — a máquina não aprova nem nega nada, o pagamento acontece na frente das duas partes. E não há reembolso: o dinheiro nunca passou pelo sistema. Ficam `PENDENTE` (a receber), `PAGO` e `CANCELADO`.

O que foi implementado

Componente	Mudança
Migration <code>008</code>	Remove colunas do gateway, muda constraints de <code>metodo</code> e <code>status</code> , muda o pagamento para por produtor
Backend	Remoção do serviço de gateway; endpoint <code>PATCH /pedidos/:id/pagamento/confirmar</code> , restrito ao agricultor
Frontend	Tela de confirmação no painel do produtor; checkout sem etapa de pagamento online
Documentação	OpenAPI, README e <code>AGENTS.md</code> alinhados; referências ao gateway removidas
Testes	Casos positivos e negativos para a confirmação; caso multi-produtor

Regras de negócio da confirmação

A operação de confirmar recebimento exige cuidado, e as regras implementadas refletem isso:

- **Quem tenta confirmar recebe códigos diferentes conforme o motivo.** Um cliente (ou admin) esbarra primeiro no `requireRole('agricultor')` e recebe `403 SEM_PERMISSAO`; um **outro agricultor** passa pelo papel e para na consulta por posse, recebendo `404 NAO_ENCONTRADO`. O `404` é o comportamento desejado — um `403` confirmaria que o pedido existe e que há pagamento para alguém.
- **A ordem das checagens importa.** Primeiro "tem pagamento meu neste pedido?" (`404` se não), e só depois as regras de estado. Invertendo, um produtor de fora receberia "pagamento já confirmado" a respeito de um pedido que não é dele.
- **Confirmar duas vezes não é erro.** O `UPDATE ... WHERE status = 'PENDENTE'` é condicional; se não encontra a linha, relemos o pagamento e respondemos com `ja_estava_pago: true`, em vez de dar baixa duas vezes.
- **Pedido cancelado não aceita confirmação.** E o cancelamento marca como `CANCELADO` apenas os pagamentos ainda `PENDENTE` — um pagamento já pago não vira cancelado por um `UPDATE` em massa.

Resultado

O checkout ficou **mais simples e mais aderente à realidade**: não há serviço externo, não há webhook, não há taxa, não há dado de cartão. O sistema passou a registrar quanto **cada** produtor tem a receber na retirada, e quem confirma o recebimento é o próprio produtor. Essa mudança responde diretamente ao setor escolhido (agricultura familiar) e ao princípio de inclusão digital: a solução tecnológica se adapta à realidade do usuário, e não o contrário.

6.2 Evolução: página "Sobre a iniciativa" (PR #8)

O link "Sobre a iniciativa" existia no cabeçalho, mas era um `<span>` sem destino. A página foi criada com conteúdo institucional sobre a iniciativa em Toledo:

- **Cabeçalho:** "Toledo, Paraná • Oeste Paranaense" e "Agro Hero Toledo: Conectando Agricultores e Consumidores";
- **Nossa Missão em Toledo — PR**, com o compromisso de 100 % do valor ao produtor;
- **Distritos atendidos:** Novo Sarandi, Concórdia do Oeste, Dez de Maio e Vila Nova;
- **Propósito:** "Mais renda para o produtor e menos desperdício".

A estilização reaproveita a linguagem visual da Home (hero verde com selo, cartões, faixas em gradiente), o que exigiu mover duas variantes de botão (`.botao--destaque`, `.botao--contorno`) do CSS da Home para um CSS compartilhado — reuso de código, ainda que o item "reuso do projeto" não tenha sido escolhido como tema.

6.3 Evolução: identidade visual do produtor (avatar e logo)

Duas funcionalidades deram identidade visual ao sistema:

- **Avatar do usuário** (migration `010`): foto de perfil exibida no cabeçalho e no perfil;
- **Logo da propriedade** (migration `009`): o produtor pode enviar a marca da sua propriedade, exibida no catálogo.

Ambas geraram aprendizados técnicos relevantes registrados no `AGENTS.md`:

- **Rota de imagem autenticada não pode ser o `src` de um `<img>`.** O navegador não envia cabeçalhos customizados ao buscar o `src` de uma imagem — ele faz a requisição simples, sem `Authorization`. Apontar `` produz 401. A solução correta (implementada em `frontend/src/services/avatar.js` + `hooks/useAvatar.js`) é buscar os bytes com `fetch` + token e montar um `URL.createObjectURL`. Token na query string e rota pública por id foram descartados por vazarem credencial / exporem a foto alheia.
- **A foto é buscada uma vez no `AuthContext`** e compartilhada entre cabeçalho e perfil — se cada tela buscasse por conta própria, trocar a foto não refletiria na outra sem recarregar.

6.4 Correção: fallback SPA no Vercel (PR #9)

Ao conferir a página "Sobre" em produção, descobriu-se que **toda rota profunda do site respondia 404**:

/produtos	/receitas	/agricultores	/categorias	/sobre
↓	↓	↓	↓	↓
404	404	404	404	404

A causa era de configuração: o projeto no Vercel estava apontado para o diretório `frontend/` mas **não tinha `rewrite SPA`**. O Vercel procurava um arquivo para o caminho pedido, não encontrava (o build só tem `index.html` e `assets/`), e respondia 404 antes de o React montar.

O defeito era **invisível navegando pelo aplicativo**: o React Router troca a URL no cliente, e nenhuma requisição é feita ao servidor. O problema só aparecia em link compartilhado, favorito ou recarregamento (F5).

A correção foi um arquivo `frontend/vercel.json`:

```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

O `rewrite` roda **depois** da checagem de arquivo existente, então os assets em `/assets/*` continuam sendo servidos normalmente; só os caminhos sem arquivo correspondente caem no `index.html`. Após o deploy, todas as rotas passaram a responder 200.

Esse defeito não estava previsto na AE III, mas é um bom exemplo de **evolução orientada por uso real**: só apareceu ao colocar a aplicação na mão de alguém que acessa por link, e não pelo menu.

6.5 Evolução de UI: remoção dos dados de infraestrutura da front page (PR #6)

Em uma iteração anterior, os dados de diagnóstico (status da API, latência, ambiente) foram exibidos na página inicial. A avaliação de uso mostrou que aquilo não deveria ficar em evidência para o visitante — é informação técnica, não conteúdo para o

consumidor. A decisão foi **remover o bloco da Home e manter a exibição apenas no rodapé**, de forma discreta, junto ao crédito do autor ("Criado por Lucas Senderski RU: 4758862").

Esse é um caso de evolução por **ajuste de design e de conteúdo**, motivado pela perspectiva do usuário final.

6.6 Rastreabilidade das evoluções

Mudança	Quem	O quê	Quando	Como	Por quê
Pagamento na retirada	Lucas (dev)	Remove gateway, cria pagamento por produtor	PR #2	Migration 008, service, painel do produtor, testes	Adequar ao público real (agricultura familiar)
Página Sobre	Lucas (dev)	Página institucional + link	PR #8	React Router, CSS compartilhado, teste	Dar identidade e contexto à iniciativa
Avatar / logo	Lucas (dev)	Identidade visual	migrations 009 / 010	Upload, fetch autenticado, contexto React	Humanizar a relação produtor-consumidor
Fallback SPA	Lucas (dev)	vercel.json	PR #9	Rewrite para index.html	Tornar as rotas acessíveis por link direto
Limpeza da Home	Lucas (dev)	Remove dados técnicos da página inicial	PR #6	Edição do componente Home	Foco no conteúdo do consumidor

7. Implantação

A implantação é obrigatória para os cursos de três Atividades Extensionistas e foi o desfecho natural desta fase. O sistema está no ar, atendendo em três serviços:

Componente	Serviço	Endereço
Frontend (React)	Vercel	https://agrohero-six.vercel.app
Backend (API)	Render	https://agrohero.onrender.com
Banco (PostgreSQL)	Neon	—

7.1 Decisões de implantação

Banco no Neon, não no Render. O plano gratuito de PostgreSQL do Render expira em 30 dias e apaga os dados, sem opção de congelamento. O Neon tem plano gratuito permanente. Isso invalidou o caminho óbvio de hospedar os três serviços no Render.

Migrations no startCommand. O Render não oferece `preDeployCommand` nem Shell no plano gratuito — exatamente as duas ferramentas de que um deploy de backend precisa. O blueprint contorna: as migrations vão no comando de início (`npm run migrate`

`&& npm start`), onde o `&&` preserva a propriedade essencial de falha segura (migration falha → servidor não sobe → versão anterior continua no ar).

`?sslmode=require` na URL do banco. É o parâmetro que liga o TLS; o driver `pg` o interpreta e configura o SSL sozinho. Provedores gerenciados recusam conexão sem TLS, e o erro que aparece é de conexão, não de SSL — o que despista.

`VITE_API_URL` é lida em tempo de build. O navegador é quem lê a variável; não há servidor para injetá-la depois. Corrigir a variável sem forçar novo deploy não muda nada — o bundle continua com o valor antigo.

7.2 Segurança em produção

A aplicação em produção roda com:

- **Documentação desativada** (`/api/v1/docs` responde 404);
- **CORS restrito** à origem do frontend (sem `*`, sem `http://`);
- **Guarda de segredos** que derruba o processo na subida se as variáveis contiverem valores de exemplo;
- **Endpoint de saúde** (`GET /health`) exposto fora do versionamento, como os provedores exigem, retornando o estado real do sistema.

7.3 Evidência de que a implantação funcionou

Um aprendizado importante registrado no projeto: **o painel do provedor diz o deploy disparado, não o que está no ar**. Um deploy que falha não derruba o anterior; a versão antiga continua servindo, e o painel ainda exibe o commit novo como mais recente. A conferência confiável é o **artefato**.

O Vite nomeia os arquivos gerados por hash de conteúdo. Portanto, reconstruir o commit candidato com a mesma `VITE_API_URL` de produção e comparar os nomes de `dist/assets` com os que o site serve prova qual commit está publicado:

```
Build local do commit → index-D6UwP-jn.js / index-Cq3M5hbm.css
O que o site serve     → index-D6UwP-jn.js / index-Cq3M5hbm.css
                        ↑ conferem: o commit publicado é o esperado
```

A conferência final, no estado atual:

Verificação	Resultado
<code>/</code> , <code>/sobre</code> , <code>/produtos</code> , <code>/receitas</code> , <code>/agricultores</code> , <code>/categorias</code>	200
<code>/health</code> da API	api: ok, banco: ok, ambiente: production
Bundle servido = build local	confere (<code>index-D6UwP-jn.js</code>)

7.4 Plano de rollback

O ponto de retorno está definido e documentado: a tag anotada `v1.0.0-producao` (commit `c2a97a3`), já publicada no repositório remoto. O procedimento está na seção 9 de

docs/DEPLOY.md. Duas distinções foram registradas como aprendizados:

- **Rollback não é ajustar código**, é *Redeploy* do commit bom no painel, ou `git revert` (não `reset --hard`, que exigiria `--force` e apagaria o commit do servidor);
- **Rollback de aplicação e de schema são decisões separadas**. Voltar o código sem voltar o schema é inconsistente: o código antigo grava colunas que a migration 008 apagou. O rollback verdadeiro de schema exigiria o *restore point* do banco.

8. Respostas ao quadro Quem / O quê / Quando / Como / Por quê

O enunciado pede que, ao final, o projeto tenha evoluído ao ponto de responder a essas perguntas para **cada atividade**. A tabela consolida as respostas para as atividades realizadas na AE III.

Atividade	Quem	O quê	Quando	Como	Por quê
Gerenciamento	Lucas Senderski	Definiu o processo em fases e a definição de pronto	Todo o período	Fases 0-24 no README, AGENTS.md como memória	Transformar o projeto em realidade com disciplina, sem depender da memória
Controle de configuração	Lucas Senderski	Versionou código, migrations e configs	Todo o período	Git + GitHub: branches, commits descritos, 11 PRs, tag de release	Rastrear mudanças e permitir rollback
Qualidade	Lucas Senderski	Escreveu e manteve 692 testes	Todo o período	Jest + Supertest, Vitest + Testing Library, sem mocks	Descobrir defeitos reais pelo caminho verdadeiro
Métricas	Lucas Senderski	Mediu SLOC/KSLOC, cobertura e tamanho de PR	Marcos do histórico	Scripts de contagem + relatório de cobertura	Acompanhar crescimento e qualidade de forma objetiva
Evolução principal	Lucas Senderski	Removeu o gateway e efetivou o pagamento na retirada	PR #2	Migration 008, service, painel do produtor, testes	Adequar o sistema à realidade do agricultor familiar
Evolução (página)	Lucas Senderski	Criou a página "Sobre a iniciativa"	PR #8	React Router, CSS compartilhado, teste	Dar contexto e identidade à iniciativa local
Evolução (imagem)	Lucas Senderski	Avatar e logo da propriedade	migrations 009 / 010	Upload com fetch autenticado, contexto React	Humanizar a relação produtor-consumidor
Correção	Lucas Senderski	Corrigiu o 404 das rotas profundas	PR #9	frontend/vercel.json com rewrite	Tornar as páginas acessíveis por link direto
Design	Lucas Senderski	Removeu dados técnicos da front page	PR #6	Edição do componente Home	Focar no conteúdo do consumidor

Atividade	Quem	O quê	Quando	Como	Por quê
Implantação	Lucas Senderski	Publicou frontend, API e banco	28/09/2026	Vercel + Render + Neon	Finalizar o projeto, colocando-o em uso

9. Conclusão

A Atividade Extensionista III foi conduzida como uma **evolução gerenciada** do AgroHero, e não como um acréscimo de funcionalidades desconectadas. Os cinco itens escolhidos se sustentam mutuamente: o processo de gerenciamento definiu o ritmo; o controle de configuração garantiu rastreabilidade e rollback; o plano de configuração delimitou o que é versionado; as métricas deram base objetiva às decisões; e a qualidade — materializada em 692 testes sem mocks — permitiu que a evolução acontecesse sem quebrar o que já funcionava.

As três conclusões que considero mais relevantes:

1. A evolução mais valiosa foi uma remoção. O maior impacto não veio de adicionar código, mas de **retirar** o gateway de pagamento e reconhecer que a solução tecnológica tinha que se dobrar à realidade do agricultor familiar, e não o contrário. O sistema ficou menor, mais simples e mais útil. Isso dialoga diretamente com a proposta da disciplina: tecnologia aplicada à inclusão digital é adequar a ferramenta a quem vai usá-la.

2. A qualidade se prova no caminho negativo. A decisão de testar sem mocks e de cobrir o que dá errado — IDOR, permissão, valores inválidos, estoque insuficiente — foi o que revelou defeitos que teriam passado despercebidos: o valor de outro produtor vazando em um pedido compartilhado, a mensagem de erro genérica escondendo a causa, o ingrediente casando dentro de outra palavra. Nenhum desses apareceria em um teste que só verifica o caso feliz.

3. O gerenciamento se prova na falha, não no sucesso. As decisões de processo mais úteis foram concebidas para o pior cenário: migration no `startCommand` para que uma falha não derrube a versão no ar; guarda de configuração que derruba o processo em vez de operar insegura; tag de release e procedimento de rollback documentado; conferência pelo artefato, porque o painel do provedor mente. Um projeto não é bem gerenciado porque tudo deu certo — é bem gerenciado porque a falha tem um caminho de volta.

O produto final está **publicado e em uso**, com o sistema acessível por link direto, API respondendo, e o banco persistente. O projeto cumpriu o ciclo completo: levantamento (AE II), estruturação (AE II), evolução e gerenciamento (AE III) e implantação (AE III).

10. Referências

GONÇALVES, Priscila de Fátima; BARRETO, Jeanine dos Santos; ZENKER, Aline Maciel; FAGUNDES, Rubem Dutra Ribeiro; ROCHA, Breno Cristovão; BIRNFELD, Karine;

TEIXEIRA, Maristela Regina Weinfurter. **Testes de Software e Gerência de Configuração**. Porto Alegre: SAGAH, 2019.

SOMMERVILLE, Ian. **Engenharia de Software**. 10. ed. São Paulo: Pearson Education do Brasil, 2018.

AGROHERO. **Repositório do projeto**. GitHub, 2026. Disponível em: <https://github.com/lucassenderski/agrohero>.

AGROHERO. **Aplicação em produção**. Vercel, 2026. Disponível em: <https://agrohero-six.vercel.app>.

11. Apêndices

Apêndice A — Pull requests integrados

PR	Título	Estado
#1	Agents/frontend style update request	Integrado
#2	feat: página de receitas + pagamento na retirada, sem gateway online	Integrado
#3	docs: registra a versão publicada após a refatoração de pagamento	Integrado
#4	feat: status da API e crédito do autor no rodapé	Integrado
#5	docs: registra a versão publicada do PR #4	Integrado
#6	fix: tira os dados de infraestrutura da front page	Integrado
#7	docs: registra a versão publicada do PR #6	Integrado
#8	feat: página "Sobre a iniciativa"	Integrado
#9	fix: fallback SPA no Vercel para as rotas do React Router	Integrado
#10	docs: registra a página Sobre e o fallback SPA no Vercel	Integrado
#11	docs: registra as armadilhas de deploy do Vercel no AGENTS.md	Integrado

Apêndice B — Migrations do banco de dados

#	Nome	Assunto
001	funcoes_base	Funções utilitárias (trigger de <code>atualizado_em</code> , etc.)
002	identidade	Usuários e endereços
003	catalogo	Agricultores, categorias, produtos
004	pedidos	Pedidos, itens, carrinho, trigger de sincronia de status
005	pagamentos_avaliacoes	Pagamentos (versão inicial com gateway) e avaliações
006	view_tipos	View <code>produtos_com_avaliacao</code>
007	redefinicao_senha	Tokens de redefinição de senha
008	pagamento_na_retirada	Remove o gateway; pagamento por produtor
009	logo_propriedade	Logo da propriedade do agricultor
010	avatar_usuario	Avatar do usuário

Apêndice C — Comandos de verificação

```
# Testes do backend (630 testes, recria o schema do zero)
cd backend && npm test

# Cobertura de testes do backend
cd backend && npm run test:coverage

# Testes do frontend (62 testes) – exige o backend em modo de teste
cd frontend && npx vitest run

# Verificação da aplicação em produção
curl https://agrohero.onrender.com/health
curl -o /dev/null -w "%{http_code}" https://agrohero-six.vercel.app/sobre
```

Nota de transparência. Este projeto foi desenvolvido por Lucas Senderski com apoio de um agente de inteligência artificial (OpenHands) na implementação, nos testes e na documentação — prática registrada no próprio histórico do repositório. O gerenciamento, as decisões de arquitetura e de negócio, a definição do escopo e a validação final são de responsabilidade do autor.